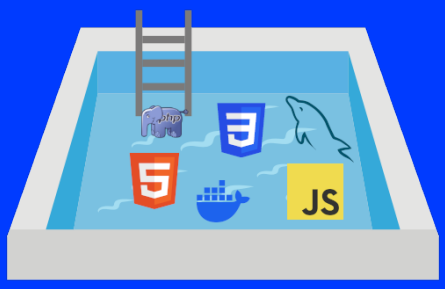
{EPITECH}

BOOTCAMP WEB_

< BONUS TASKS FOR FAST LEARNERS />



BOOTCAMP WEB

These tasks sit outside the ten-day timeline: nothing here is required to finish the bootcamp. Pick up a section whenever you are ahead of the group or come back to it afterwards.

Each section is handed in to its own directory, so the three of them never collide:

- ✓ `dayBonus/javascript/` for **JavaScript for web - extra tasks**;
- ✓ `dayBonus/sql_basics/` for **SQL - Basics**;
- ✓ `dayBonus/sql_advanced/` for **SQL - Advanced**.

{EPITECH}

JavaScript for web - extra tasks

Let's go see behind the curtain and create some JavaScript logic for provided HTML pages.



Like a previous day, some HTML files are provided and used to (auto) correct your exercises.
You have to interact with the HTML code given in order to solve the tasks.
You have to make sure that your JavaScript code does NOT require editing the HTML code.



Today, JavaScript is much more powerful than when jQuery came out, **17 years ago**.
And remember that **simple is beautiful!**



Of course, using any external library or code copied from the Internet is forbidden.

{EPITECH}

Tasks

Use your JavaScript skills directly from your favorite web-browser to solve each task.
The tasks are provided as HTML pages. You have to:

- ✓ interact with the provided HTML code;
- ✓ hand in some JavaScript files located in a `script` folder;
- ✓ respect the naming convention.



Editing the HTML code is not necessary.
You don't have to submit the provided HTML files in your delivery.

For each task, you'll find below the naming convention for the resources and your delivery.

Tasks	Resource	Delivery
task01	task01.html	day-Bonus/javascript/scripts/task01.js
task02	task02.html	day-Bonus/javascript/scripts/task02.js
task03	task03.html	day-Bonus/javascript/scripts/task03.js
task04	task04.html	day-Bonus/javascript/scripts/task04.js
task05	task05.html	day-Bonus/javascript/scripts/task05.js
task06	task06.html	day-Bonus/javascript/scripts/task06.js
task07	task07.html	day-Bonus/javascript/scripts/task07.js
task08	task08.html	day-Bonus/javascript/scripts/task08.js
task09	task09.html	day-Bonus/javascript/scripts/task09.js



Not respecting naming standards can hurt, lead to un-evaluation and increased errors.

{EPITECH}


with the naming convention is mandatory!
Complying with the naming convention is mandatory!
Complying with the naming convention is mandatory!
Complying with the naming convention is mandatory!
Complying with the naming convention is mandatory!
Complying with the naming convention is mandatory!
Complying with the naming convention is mandatory!
Complying with the naming convention is mandatory!

SQL - Basics

Let's become familiar with relational databases and the language they speak: SQL.

There are many database management systems (DBMS): Microsoft SQL Server and Oracle MySQL for commercial DBMS, and SQLite, PostgreSQL, MariaDB for free and open DBMS.

For practical reasons, we will use SQLite which is free and is also one of the most widely used DBMS.



There are more widely used DBMS like PostgreSQL or MySQL but they require a bit more setup. But don't worry, you'll use these later, and every SQL request you write today will work just as well with these.



Every task can technically be solved with the help of a Generative AI (like ChatGPT or Copilot).
But don't do it.
This workshop is meant to help you **understand how SQL really works** — not just how to ask a tool for an answer.
If you skip that learning step, you won't be able to:

- ✓ read or debug real queries on your own,
- ✓ understand why a query gives a certain result,
- ✓ combine multiple SQL concepts when the problem isn't already solved online,
- ✓ or even detect when an AI gives you a *wrong* or *nonsensical* query (and it happens a lot).

The goal is not just to ”get the output” — it's to build the reflexes and logic of how data is selected, filtered, sorted, and grouped.
That's something **no AI can learn for you**.
So, **use your brain before your tools.**
You'll thank yourself later when you're debugging real code in a project or internship.



You should take note that the database that will be used for testing does not contain the same data as the one given here. Which means that you shall only use general request and no request like ”id = 42” for example.

Set up

To use SQLite, you don't need to set up any server.
SQLite is a lightweight, file-based database system that works the same on all operating systems.

Install SQLite

- ✓ On **Ubuntu / Linux**, run the command `sudo apt-get install sqlite3`
- ✓ On **macOS**, run the command `brew install sqlite`
- ✓ On **Windows**, download the precompiled binaries from the official website. Then, Extract them and make sure the folder containing `sqlite3.exe` is in your PATH (or simply run it from the same directory in your terminal).



On **Windows** it may be easier to just boot up your WSL installation (remember, the one you used on day01?) and install SQLite like if you were on Linux.

Load the database You don't need to create or import anything manually. A ready-to-use database file named `movies.db` is provided with the project. Simply make sure it's in the same directory as your project files.

To open the database with SQLite, run `sqlite3 movies.db`. You can now explore its content:

```
sqlite> .tables
sqlite> SELECT * FROM movies LIMIT 5;
```

To exit SQLite, type:

```
sqlite> .exit
```

You should be set up and ready to write some awesome SQL queries.



For every task you'll have to write the correct SQL query in a file to perform the required action.
To test your query, use SQLite as shown above.
A useful resource: https://www.w3schools.com/sql/.

{EPITECH}

Task01

Delivery `./dayBonus/sql_basics/task01.sql`

Display the **title** and **summary** of all the movies sorted in alphabetical order.

Task02

Delivery `./dayBonus/sql_basics/task02.sql`

Display the **name** of all the genres in the table **genres** in uppercase.
The column will be named ”NAME OF ALL THE GENRES”.

Task03

Delivery `./dayBonus/sql_basics/task03.sql`

Display the **title** of the last 42 movies in the table **movies**. The column will be named ”Title of the last 42 movies”. The results must be ordered by decreasing id.

Task04

Delivery `./dayBonus/sql_basics/task04.sql`

Display the **name** and the **price** of the most expensive subscription in the **subscriptions** table.
The columns will be respectively named: ”Name of the most expensive subscription” and ”Price”.

Task05

Delivery `./dayBonus/sql_basics/task05.sql`

Display the movie's title whose genre is ”action” or ”romance”



The goal here is to use the **JOIN** operation.

{EPITECH}

Task06

Delivery `./dayBonus/sql_basics/task06.sql`

Display how long the shortest movie is in minutes.
The movies with NULL or 0 duration should not be taken into account.
The column will be named ”Duration of the shortest movie”.

Task07

Delivery `./dayBonus/sql_basics/task07.sql`

Display the id of the movies whose title contains the chain of characters ”tard” (case-insensitive).
The column will be named ”Identifier”.

Task08

Delivery `./dayBonus/sql_basics/task08.sql`

Display the number of movies whose title ends up with the string ”tion” (case-insensitive).
The column will be named ”Number of movies ending with ”tion””.

Task09

Delivery `./dayBonus/sql_basics/task09.sql`

Display the total number of movies whose genre is ”western” and whose producers are ”tartan movies” or ”lionsgate uk”.
The column will be named ”Number of 'western' movies”.

Task10

Delivery `./dayBonus/sql_basics/task10.sql`

Display the number of a room and its name for the rooms that have more than 0 seats and that are not on the first floor (First floor and not ground floor).
The columns will be named ”Room numbers” and ”Room names”.

{EPITECH}

Task11

Delivery `./dayBonus/sql_basics/task11.sql`

Display the number of movies whose title starts with ”eX” (case-sensitive).
The column must be named 'Number of movies that start with ”eX”'.

Task12

Delivery `./dayBonus/sql_basics/task12.sql`

Display the average duration of a movie rounded to 2 decimals.
The column will be named ”Average duration”.

Task13

Delivery `./dayBonus/sql_basics/task13.sql`

Display the month of birth in English from the 42nd to the 84th member (the 42nd and the 84th must be included).
The column will be named ”month of birth”.



You may need to use **CASE**.



You have to use the **JOIN** operation.

Task14

Delivery `./dayBonus/sql_basics/task14.sql`

Display the **title** of the longest movie.
The column will be named: ”Title of the longest movie”.

{EPITECH}

Task15

Delivery `./dayBonus/sql_basics/task15.sql`

Display the **lastname** followed by a dash, followed by the **firstname** of each member from the table **profiles**. The first letter of the last name and the first name's first letter will be in upper case. The members should be displayed from the youngest to the oldest.
The column will be named: ”Full name”.

Task16

Delivery `./dayBonus/sql_basics/task16.sql`

Display the **title** of the movies whose **id** is 21, 87, 263, 413 or 633.
The column will be named ”Movie title”.

Task17

Delivery `./dayBonus/sql_basics/task17.sql`

Display the number of produced movies per year. The year must not be 0.
The result has to be ordered by decreasing year of production.
The columns will be named ”Number of movies” and ”Year of production”.

Games



If you need a little break in your day, we suggest you to play:

- ✓ SQL island;
- ✓ SQL noir;
- ✓ SQL Police Department;
- ✓ Lost at SQL;
- ✓ The SQL Murder mystery;
- ✓ Lord of the SQL injections.

We also encourage you to resume your game(s) later.

{EPITECH}

SQL - Advanced

Games



If you need a little break in your day, we suggest you to play:

- ✓ SQL island;
- ✓ SQL noir;
- ✓ SQL Police Department;
- ✓ Lost at SQL;
- ✓ The SQL Murder mystery;
- ✓ Lord of the SQL injections.

We also encourage you to resume your game(s) later.

Task01

File to turn in: `./dayBonus/sql_advanced/task01.sql`

Display the number of members and their average age, rounded to the whole number (although the result will still be a floating-point value).
The columns will be named ”Number of members” and ”Average age”.
A man of 55 years and 11 months is considered to be only 55 years old.

Task02

File to turn in: `./dayBonus/sql_advanced/task02.sql`

Display only the **zip code**s where there is more than one individual.
Display them in ascending order.
The column will be named ”Zip codes”.

Task03

File to turn in: `./dayBonus/sql_advanced/task03.sql`

{EPITECH}

For each floor, display its number, its total number of seats and its total number of rooms.
The columns will be named ”Floor number”, ”Total number of seats” and ”Total number of rooms”.
Sort everything by ascending number of seats.

Task04

File to turn in: `./dayBonus/sql_advanced/task04.sql`

Display the first 92 characters of the **summary** for movies with odd **id**s between 42 and 84.
The column will be named ”Summaries”.

Task05

File to turn in: `./dayBonus/sql_advanced/task05.sql`

Display the **email** addresses of the members in the table profiles, replacing the string ”machin.com” by ”epitech.eu”. The whole list will be sorted by reverse alphabetical order.
The column will be named ”New email addresses”.

Task06

File to turn in: `./dayBonus/sql_advanced/task06.sql`

Display for each movie their **title** and the number of days since they were released.
The columns will be named: ”Movie title” and ”Number of days passed”.
The release date must be defined.

Task07

File to turn in: `./dayBonus/sql_advanced/task07.sql`

Display the **title** of movies for which the first letter of their title is between ”O” and ”T” included.
The whole list has to be sorted in alphabetical order.
The column will be named ”Movie title”.

{EPITECH}

Task08

File to turn in: `./dayBonus/sql_advanced/task08.sql`

Display the **name** of the table genres whose **id** is not between 6 and 12.
Those numbers should not be included in the final result.

Task09

File to turn in: `./dayBonus/sql_advanced/task09.sql`

Display the **title** and the **min_duration** of all the movies.
The result has to be sorted by descending the length of the title and then by ascending the movie duration.

Task10

File to turn in: `./dayBonus/sql_advanced/task10.sql`

Sum the **prod_year** of all the movies in a column ”Sum prod_year”.
Each year must be summed only once.

Task11

File to turn in: `./dayBonus/sql_advanced/task11.sql`

Add a new entry in the subscription table.
Its name will be ”Premium” and its summary will be: ”For the privileged”.
Its price 80 and its duration of subscription 126.

Task12

File to turn in: `./dayBonus/sql_advanced/task12.sql`

Go back to the query of the previous exercise and add the premium subscription 5 times.
You must only use one INSERT INTO statement.

{EPITECH}

Task13

File to turn in: `./dayBonus/sql_advanced/task13.sql`

Delete the last 4 subscriptions of the subscription table.

Task14

File to turn in: `./dayBonus/sql_advanced/task14.sql`

Update the name of the last subscription to ”Premium++”.

Task15

File to turn in: `./dayBonus/sql_advanced/task15.sql`

Update the email addresses of members by replacing the string ”machin.com” with ”epitech.eu”

Task16

File to turn in: `./dayBonus/sql_advanced/task16.sql`

Delete the movies whose **prod_year** is equal to 0.

Task17

File to turn in: `./dayBonus/sql_advanced/task17.sql`

Update the field **producer_id** in the movie table.
When the field is not defined, set it to the id of the producer who has the fewer movies.
Moreover, the producer name must finish by ”film”.



Task18

File to turn in: `./dayBonus/sql_advanced/task18.sql`

Delete all movies in the movie table that were released in 1990.

Task19

File to turn in: `./dayBonus/sql_advanced/task19.sql`

Add in the **job** table the list of jobs provided in the CSV file.
The file will be in the same format and will be in ”/tmp/jobs.csv”.



For this task, you’ll need to include multiple commands in the delivery file.
These commands can be a combination of SQL queries and SQLite-specific commands.

Task20

File to turn in: `./dayBonus/sql_advanced/task20.sql`

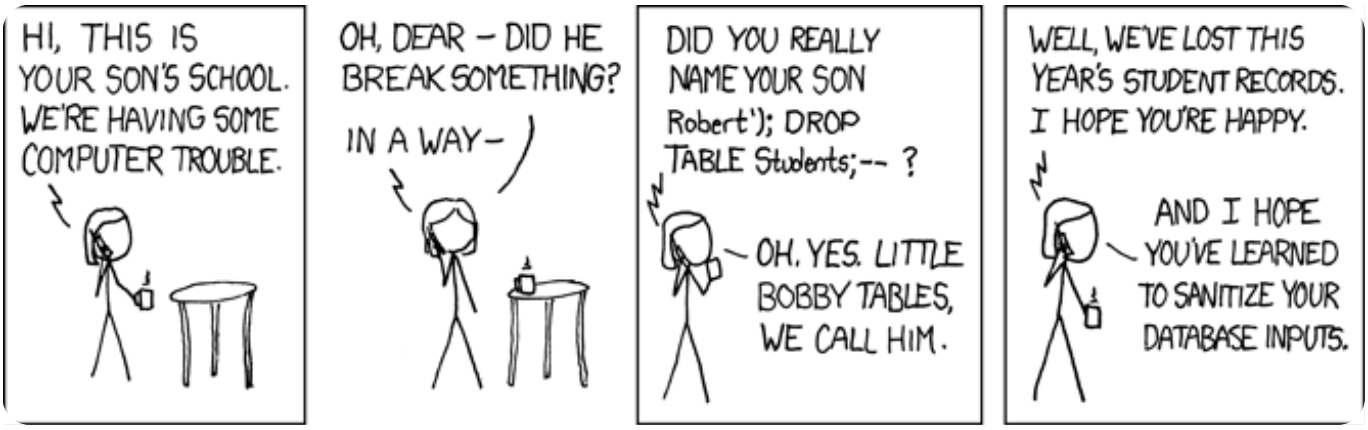
Export the content of the movie table with format CSV in a file in /tmp/movies.csv.



For this task, you’ll need to include multiple commands in the delivery file.
These commands can be a combination of SQL queries and SQLite-specific commands.

{EPITECH}

A little bit of fun



> I quit my partner because he has a one-to-many relationship.
> — *a database administrator*

> Why do front end developers eat lunch alone?
> Because they don't know how to join tables.
> — *a backend developer*

{EPITECH}

v1

